

# Private Lending for Aleo

A native Leo implementation of PWN's bilateral, oracle-free lending protocol.

| APPLICANT                                                        | TIMELINE                            | GRANT REQUEST                            | DELIVERABLE                                                  |
|------------------------------------------------------------------|-------------------------------------|------------------------------------------|--------------------------------------------------------------|
| <b>PWN DAO Foundation</b><br>P2P lending live on 13 EVM mainnets | <b>~4 months</b><br>four milestones | <b>\$85,000</b><br>capped mainnet launch | <b>Protocol on mainnet</b><br>programs · SDK · reference app |

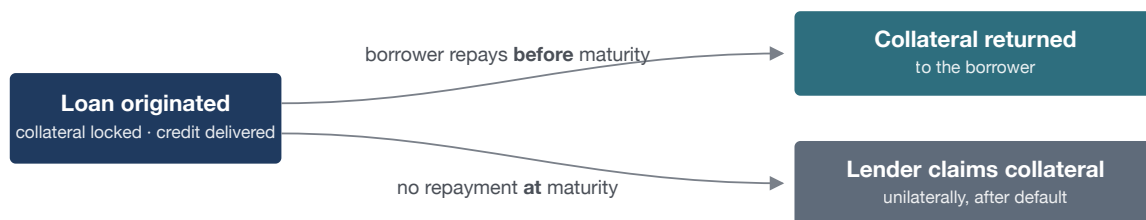
The applicant is the **PWN DAO Foundation**, which operates a peer-to-peer lending protocol live on 13 EVM mainnets. Lenders and borrowers agree on fixed loan terms directly, without price oracles or price-based liquidations. This bilateral model fits a privacy-first ZK chain particularly well, because individual loans stay isolated instead of depending on global liquidity, utilisation and liquidation state.

This grant funds a native Aleo implementation of that protocol: funded Elastic Proposals, private bilateral loans, a modular Hub and Config layer, a TypeScript SDK and a reference application, **with a capped mainnet launch as the final milestone**. A feasibility PoC of the core programs is already deployed and exercised on the Aleo testnet (see *Current status*).

## Protocol model

Each loan fixes the credit asset and amount, the collateral asset and amount, the repayment amount and the maturity. If the borrower repays before maturity, collateral is returned. Otherwise the lender can claim it after default. There is no price feed, no health factor and no liquidation: nobody ever needs to inspect anyone else's position, which is what makes the model a natural fit for a chain where state is private by default.

The Aleo implementation is a native Leo adaptation of PWN, not a Solidity port.



Two outcomes, decided only by the borrower's action and the clock. No third party, no price feed, no liquidation.

---

## Current status

The feasibility work is done. The four core programs are deployed on the public Aleo testnet and the complete lifecycle (funded commitment, borrower draw, repayment, default claim, refund and repayment withdrawal) has been executed there with real proofs, collateral on the live `token_registry.aleo`, and privately paid fees.

### Custody & funded commitments

#### Demonstrated on testnet.

Lender commits once; borrower draws without any lender transaction; every exit settles into a private record.

### Privacy

#### Measured on real transactions.

No lender or borrower address in any lending transaction; amounts, terms and IDs public by design.

### Costs & timings

#### Within limits. 0.005–0.019

ALEO per call; proving 25–41 s on a laptop; confirmation in 1–4 blocks.

Not yet done: direct offers, launch caps, USDCx execution against the live stablecoin, a complete test suite and external review. These are the subject of the milestones below.

---

## Feasibility questions, answered by the PoC

- |    |                                                                                                                                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 01 | <b>Can lender credit be committed once and drawn asynchronously without lender participation?</b> Yes. Draws are signed by the borrower only.                                                                                |
| 02 | <b>Can custody safely support private → controlled state → private asset flows?</b> Yes, for native credits and token-registry assets; a premature default claim was rejected by network finalization.                       |
| 03 | <b>What is public, private, inferable or linkable in each transition?</b> Measured; see the privacy page.                                                                                                                    |
| 04 | <b>Does the model work within USDCx compliance and token-registry constraints?</b> Token registry: yes, live. USDCx: interfaces compile against the real bytecode; execution with freeze-list proofs is part of Milestone 1. |
| 05 | <b>Can Proposal, Loan and token programs compose within proving and transaction limits?</b> Yes. A draw is four transitions across three programs in one transaction, at about 6% of the constraint limit.                   |
| 06 | <b>Measured proving and execution times?</b> 25–41 s per call on a laptop; 0.005–0.019 ALEO per call; confirmation in 1–4 blocks.                                                                                            |

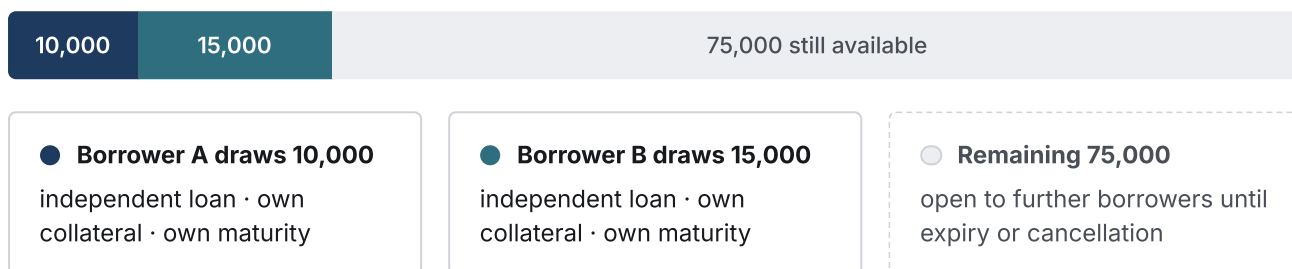
---

## Elastic Proposal: the default offer type

In PWN's EVM implementation, an [elastic proposal](#) lets one lender publish a single reusable offer with a credit limit, a minimum draw and a fixed credit-to-collateral ratio. Borrowers draw from it independently; each draw is its own bilateral loan with its own collateral and maturity. It is not a pool: no shared position, no mutualised risk.

Lender commits **100,000 USDCx**

min draw **5,000**



### Why this is the default on Aleo

An Aleo transaction has a single signer and a program cannot spend a private record on a counterparty's behalf, so asynchronous acceptance requires one side to commit capital in advance. Elastic proposals assume exactly that, and bring three things for free: every offer is backed by real capital, offers live in public state so discovery needs no PWN backend, and many draws against one commitment make individual loans harder to link to a lender. Direct offers use the same mechanism with a restricted acceptor.

---

## Discovery and indexing

Public Elastic Proposals remain canonical in Aleo public state. The SDK queries that state directly; an optional indexer or cache improves filtering and UX but is never a trusted source of truth. Private user loan records require a separate record-discovery strategy at the wallet layer.



---

## Architecture

| LAYER             | PROGRAM                         | CONTROLLED BY                  | CAN AFFECT EXISTING LOANS                                |
|-------------------|---------------------------------|--------------------------------|----------------------------------------------------------|
| Governance        | —                               | PWN DAO<br>Foundation multisig | No                                                       |
| Registry & config | pwn_hub.aleo<br>pwn_config.aleo | multisig                       | No: approves modules for<br><i>new</i> originations only |
| Proposal          | pwn_elastic_proposal.aleo       | lender                         | Own commitment only                                      |
| Loan core         | pwn_loan.aleo                   | protocol logic                 | Terms fixed at origination                               |
| Integration       | TypeScript SDK                  | anyone                         | —                                                        |
| Application       | Reference app, third<br>parties | anyone                         | —                                                        |

**Governance can decide what may be created next.** It can never alter, seize or redirect a loan that already exists.

**Hub.** Registers approved modules for new loan originations: `pwn_loan.aleo` → `ACTIVE_LOAN`, `pwn_elastic_proposal.aleo` → `PROPOSAL_MODULE`. Future installments, crowdfunding and refinancing arrive as new versioned modules.

**Config.** Minimal protocol-wide settings: protocol fee (zero in V1), fee recipient, multisig authority, launch caps, and an emergency stop for new originations. Loan-specific economic terms are fixed at creation and cannot be changed later through Config.

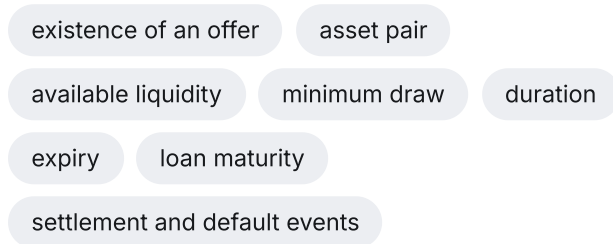
**Upgradeability.** Extensibility and bug remediation are separate concerns. New functionality is introduced as new Hub-approved modules; upgradeability, if enabled, is only a temporary safety mechanism for bug remediation, and custody and settlement logic capable of causing loss of funds should become immutable after audit and stabilisation.

## Privacy model, measured on testnet

V1 behaviour as measured on the public Aleo testnet with the PoC programs (September 2026), from confirmed transactions re-fetched from the network. What remains linkable is listed, not hidden.

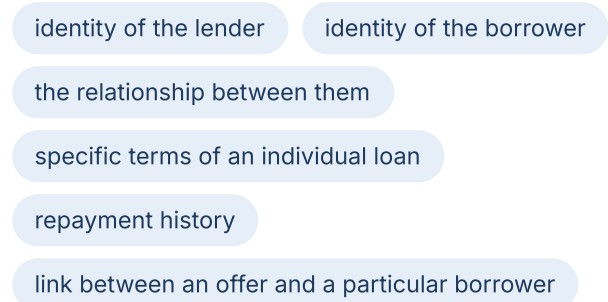
### PUBLIC BY DESIGN

Required for the protocol to function, not a compromise.

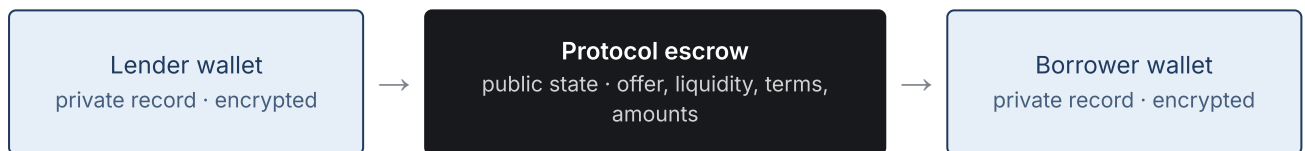


### PRIVATE IN V1

Never written to protocol public state.



### WHERE THE BOUNDARY SITS



Value passes through program-controlled public state; ownership on both ends stays in encrypted records. Amounts are visible in the middle; that is the leakage measured below.

## Known leakage, measured

| LEAK                             | MEASURED ON TESTNET                                                                                                                                            | MITIGATION                                                                        |
|----------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <b>Collateral amount</b>         | Public in the loan witness and in the token escrow delta                                                                                                       | Normalised denominations; commitment-based private accounting in a later version  |
| <b>Draw amount</b>               | Public in the loan witness; delta in available liquidity per draw                                                                                              | Normalised draw steps                                                             |
| <b>Lender / borrower address</b> | <b>Not present</b> in any of the six lending flows when fees are paid from a private record; only program addresses appear. A public fee would name the payer. | Private fees, validated; the earlier public shielding transaction remains visible |
| <b>Commitment → loan link</b>    | Public by construction (commitment ID inside the loan witness)                                                                                                 | Elastic proposals dilute it: many draws instead of one                            |
| <b>Temporal correlation</b>      | 88 blocks between the borrower's shielding and the draw in the test run                                                                                        | Documented rather than concealed                                                  |

---

## Milestones

The feasibility gate has already been passed with the testnet PoC. The grant funds the path from PoC to a reviewed, capped mainnet launch: hardening and completing the core programs, an SDK, a usable application, and a security review.



- 
- 1**

**Core protocol completion & interface freeze**

**\$30k**  
6 weeks

**SCOPE** The core of the project: direct offers, protocol-level launch caps and emergency stop in Config, USDCx execution against the live stablecoin (freeze-list proofs, pause behaviour), a full test suite covering every state transition and failure path, adversarial tests, documented invariants, and agreement of the data model and program interfaces.

**VALIDATION** On the public testnet: funded proposal with at least two independent borrowers; direct offer works; expiry, cancellation, overdraw, early-claim, double-settlement and replay protections pass; Hub and Config changes provably do not affect existing loans; one USDCx loan opened, drawn and repaid, or the blocker documented; test suite and invariants published.
  - 2**

**SDK & discovery**

**\$15k**  
3 weeks

**SCOPE** TypeScript SDK, public proposal discovery from Aleo state, private record handling, fee strategy (private fees by default), USDCx integration.

**VALIDATION** SDK supports discover, open, accept, repay, claim, cancel and withdraw; discovery works without a proprietary PWN backend; an integration test suite runs against the testnet deployment.
-

---

### 3

#### Reference application

\$20k

4 weeks

**SCOPE** Lender and borrower application on top of the SDK: wallet connection, offer publishing and browsing, draw, repayment, default claim, position overview from private records, selective disclosure of a loan to a chosen party.

**VALIDATION** External testers complete the full lender and borrower journeys on testnet without help from the team; usability findings addressed; the application is published with usage documentation.

---

### 4

#### Security review & capped mainnet launch

\$20k

4 weeks

**SCOPE** Threat model, freeze of custody and settlement logic, external review or scoped audit of the core programs, mainnet deployment with caps, SDK published as a package.

**VALIDATION** All critical invariants covered by tests; external reviewer engaged or a documented audit scope and quote; at least one real loan originated, repaid and settled on mainnet end to end by an external user; deployment addresses published.

**Capped launch.** Mainnet deployment launches with protocol-level caps on maximum commitment size and total value locked, lifted only after the security review completes. This allows real usage from day one while bounding worst-case loss.

**If Milestone 1 finds a blocker that cannot be resolved,** for example in USDCx compliance behaviour, we publish the findings and the privacy analysis as an open report for the ecosystem, and the remaining milestones are not claimed.

---

## Open questions

### Upgradeability

Keep selected programs upgradeable for three to six months as a bug-remediation grace period, or launch core financial logic as non-upgradeable from the start? Can upgrade authority be constrained so it cannot seize or redirect existing user funds?

---

### Licensing

Does the Developer Grants Program require a specific open-source licence such as Apache-2.0, or is GPL-3.0 acceptable?

---

## Collateral policy

A token issuer with a public-burn role can burn a public balance from any address that holds it, including a loan escrow, without the holder's consent; private records cannot be burned by the issuer, only by their owner. The core currently accepts any registry token as collateral. Should V1 enforce a collateral whitelist in the contracts, or keep the contracts asset-agnostic and leave the policy to the SDK and application layer?

---

## Out of scope

V1 is deliberately narrow: bilateral loans with terms fixed at origination. No oracles, no price-based liquidation, no pooled lending, no bridging to PWN's EVM deployments, no KYC or credit scoring. Installments, crowdfunded funding and refinancing are planned extensions. Protocol fees are zero. V1 targets a capped mainnet launch; the extensions follow later.

## After the grant

The deployed protocol becomes part of the PWN DAO Foundation's maintained infrastructure and is operated alongside the existing EVM deployments, under the same governance. The SDK is published as an open-source package for third-party integrators.

---

## Possible extensions

Installment loans

Crowdfunded funding: several lenders fund one loan, each holding a private position

Refinancing

Private collateral amounts

---

## Existing references

### **PWN Protocol v1.5**

[github.com/PWNDAO/pwn\\_protocol](https://github.com/PWNDAO/pwn_protocol)

Architecture and lending-model reference: Hub, Config, loan lifecycle, Elastic Proposals, installments and crowdfunding concepts.

---

### **PWN Elastic Proposal**

[dev-docs.pwn.xyz/smart-contracts/core/smart-contract-reference/proposals/simple-loan-proposal/elastic-proposal](https://dev-docs.pwn.xyz/smart-contracts/core/smart-contract-reference/proposals/simple-loan-proposal/elastic-proposal)

Reference semantics of the elastic proposal: credit limit, minimum draw, credit-to-collateral ratio, expiry and per-draw loan creation, as implemented on EVM.

---

### **ProvableHQ compliant stablecoin / USDCx**

[github.com/ProvableHQ/compliant-stablecoin](https://github.com/ProvableHQ/compliant-stablecoin)

Public and private token flows, freeze-list compliance, role administration, multisig upgrades, USDCx integration.

These are design references; the Aleo programs are written from scratch in Leo. Licensing is checked separately.